As discussed in chapter 6, the advantage of having two separate pieces of middleware to handle this process is that any middleware placed after the `RoutingMiddleware` can see which endpoint is going to be executed before it is. You'll see this benefit in action when we look at authorization in chapter 24.

Routing in ASP.NET Core uses the same infrastructure and middleware whether you're building minimal APIs, Razor Pages, or MVC controllers, but there are some differences in how you define the mapping between your route templates and your handlers in each case. In section 14.2 you'll learn the different approaches each paradigm takes.

14.2 Convention-based routing vs. explicit routing

Routing is a key part of ASP.NET Core, as it maps the incoming request's URL to a specific endpoint to execute. You have two ways to define these URL-endpoint mappings in your application:

- Using global, convention-based routing
- Using explicit routing, where each endpoint is mapped with a single route template

Which approach you use typically depends on whether you're using minimal APIs, Razor Pages, or MVC controllers and whether you're building an API or a website (using HTML). These days I lean heavily toward explicit routing, as you'll see.

Convention-based routing is defined globally for your application. You can use convention-based routes to map endpoints (MVC controller actions